Kartik Handoff: GreenOps Devices and Features

Oct 22, 2026 · @minnie

Kartik, you now own the whole device restructure for Phase 1: data check, specs, device formulas, plots, two quick models and a site scorecard, in about 3 hours, with the order and cut-offs below.

Rules and context

Why this exists. After the Phase 1 review, the faculty said our "awareness" and features were too generic. The fix is to make everything device-specific: a solar lamp, a solar water heater and a small wind turbine each get their own features, their own weather estimate and their own answer for the user (lit hours tonight, hot-water temperature, turbine kWh). The full plan is in GreenOps Data, EDA, Feature and Device Plan; this page is the short version you can work from.

Ground rules

- Weather is real (NASA POWER, 8 Karnataka sites). Device output is simulated from formulas. Every slide and plot with device numbers says "simulated".
- NASA uses -999 for missing. Turn it into NaN; never drop the row or fill it silently.
- Features for day t use days up to and including t only. The label is day t+1 ("tomorrow").
- Split by time: train up to 2023-12-31, validation 2024, test 2025-01-01 onward. Choose anything (model, threshold, setting) on validation. Touch test once.
- Any threshold (such as the low-wind cut) is fitted on train only and then frozen.
- Work on a branch, not main. Don't touch the DVC remote today.
- If a result is bad or odd, report it as it is. A model that does not beat persistence is a valid result.

What you hand back is listed at the end of this page. Minnie is busy, so you also write the one-line "What the data shows / Inference" text under each plot.

Order of work and time boxes

Two scripts do most of the work: `devices.py` (formulas and physics tests) and `run_all.py` (loads the data, makes plots, trains two models, scores devices, writes the scorecard). Both are in the "Code" section at the end of this page. I ran them end to end on made-up NASA-format data, so the logic works, but they have not seen your real files. Expect to fix column or file-name issues in the first 10 minutes.

- [ ] **Step 1, 10 min:** put the NASA files in one folder, set `NASA_GLOB` and the site names in `run_all.py`, and run it. It stops with a clear message if dates are missing or sunlight is negative.
- [ ] **Step 2, 25 min:** device specs. Pick one real lamp kit, one solar water heater and one turbine up to 10 kW, and replace the placeholder numbers in `SPECS` with sourced values.
- [ ] **Step 3, 15 min:** run `python devices.py` (physics tests must pass), then look at one site's lamp, heater and turbine output.
- [ ] **Step 4, 15 min:** look at the four EDA plots in `results/` and write the two lines under each.
- [ ] **Step 5, 30 min:** check the model results; fix anything odd; note which planner wins and by how much.
- [ ] **Step 6, 20 min:** read the device error table and the site scorecard; write the sample awareness messages.
- [ ] **Step 7, 20 min:** write `RESULTS.md` and send the files (list at the end).
- [ ] **Buffer, 30 min** for the things that go wrong.

If you run short, cut in this order: the scorecard and recommendations, then the heater part of the device error, then EDA plots 2 and 4, then the Gradient Boosting model. Never cut the specs sources, the physics tests, or the time split.

Do not run the final numbers with placeholder specs. The scripts print a warning until `SPECS_VERIFIED = True`. Any number produced before that is a test number, not a result.

Step 1: load the NASA data and check it (10 min)

1. Make a branch and a folder, for example `data/raw/nasa/`, and copy the 8 NASA POWER daily files into it, one file per site. The file name (without `.csv`) becomes the site name, so name them `Pavagada.csv`, `Raichur.csv`, `Kalaburagi.csv`,

`Mysuru.csv`, `Chitradurga.csv`, `Gadag.csv`, `Koppal.csv`, `Hassan.csv`, or edit the site lists at the top of `run_all.py` to match your names.

2. Each file must have the columns `YEAR, MO, DY, ALLSKY_SFC_SW_DWN, T2M, WS10M, WS50M`. The loader skips any header lines above the line that starts with `YEAR`. If your files hold different column names, change them in `load_nasa`.
3. Run `python run_all.py`. The first printout shows rows per site (expect 2,528 each) and NaN counts per column.
4. The script stops if a site has a missing or duplicate date, or negative sunlight. Fix the file; don't loosen the check.

What to write down for the dataset slide

- Rows per site and the number of sites (8 sites x 2,528 days = 20,224 rows, if it matches).
- Missing values per column, and that -999 was turned into NaN.
- Date range, and the split: train to 2023-12-31, validation 2024, test 2025-01-01 onward.
- Which sites are solar (lamp, heater) and which are wind (turbine).

If the real numbers differ from the ones above, use the real ones and tell Minnie.

Step 2: device specs with sources (25 min)

The numbers in `SPECS` inside `devices.py` are placeholders I typed to test the code. They are not sourced and must be replaced. Pick one real product per device, copy the values from its datasheet or a government page, and record where each came from.

Device	Parameter in `SPECS`	What it is	Where to look
Lamp	`panel_W`	solar panel rating in watts	product datasheet
Lamp	`battery_Wh`	battery energy (volts x amp-hours)	product datasheet
Lamp	`bulb_W`	bulb power	product datasheet
Lamp	`hours_wanted`	hours of light the user needs per night	the product's stated run time
Lamp	`depth_of_discharge`	usable share of the battery (0 to 1)	battery datasheet

Device	Parameter in `SPECS`	What it is	Where to look
Lamp	`temp_loss`	panel power loss per degree C above 25 (about 0.004 means 0.4 percent per degree)	panel datasheet, "temperature coefficient of power"
Lamp	`system_eff`, `charge_eff`, `temp_rise`	wiring and controller efficiency, charging efficiency, cell temperature above air	often not on a datasheet: mark as assumption and say why
Heater	`area_m2`, `tank_litres`	collector area and tank size	product datasheet or MNRE-listed model
Heater	`collector_eff`	share of sunlight turned into heat	test report or datasheet; mark as assumption if not found
Heater	`UL0`, `UL1`	tank heat loss in watts per degree C, and its increase per m/s of wind	datasheet standby loss if given; otherwise assumption
Heater	`setpoint_C`, `T_max_C`	hot-water target and the safe maximum	the product's stated temperature
Turbine	`rated_kW`, `cut_in`, `rated_speed`, `cut_out`	rated power and the wind speeds (m/s) where it starts, reaches rated power and shuts down	manufacturer datasheet or power curve
Turbine	`hub_height_m`	tower height	datasheet; if not given, use the height stated for the example site and say so

Record the sources. Save `device_specs.csv` with the columns `device, parameter, value, unit, source_type, source_url, accessed_on, is_assumption, note`. Allowed `source_type`: datasheet, standard, government_page, assumption. Every row needs a unit and a source_type. Every assumption needs a note on why you chose it.

When done: copy the values into `SPECS`, set `SPECS_VERIFIED = True`, and rerun `python devices.py`. If you could not source a value, leave it as an assumption in the CSV and keep `SPECS_VERIFIED = False`; the slides must then say "assumed values".

Turbine check. Pick a turbine up to about 10 kW. NASA gives wind at 10 m and 50 m, and the code reaches the hub height by interpolation, so the hub height should be between 10 m and 50 m.

Step 3: the device script and its tests (15 min)

`devices.py` holds the three device models and the physics tests. It is in the Code section. Run `python devices.py`; it must print "all device tests passed".

What each function does, in words

- **`lamp_simulate(sun, temp)`**: each day the panel charges the battery (panel watts x sunlight hours x efficiencies, less a hot-panel loss), the lamp draws power for the hours wanted, and the battery never goes below its floor. The battery carries into the next day, so a cloudy streak drains it. Output: lit hours, battery level, and a "fails" flag when the lamp lights less than the hours wanted.
- **`heater_simulate(sun, temp, wind10)`**: heat gained = collector area x efficiency x sunlight. Heat lost rises with the tank temperature above the air and with wind. The net heat warms the tank water. Output: hot-water temperature and a "backup needed" flag when it is below the setpoint. The loss uses the setpoint as the tank temperature, which is a simplification; say so on the slide.
- **`hub_speed(ws10, ws50, hub_height)`**: the wind speed at the turbine's height, using the shear between 10 m and 50 m.
- **`daily_energy_kwh(v_mean)`**: turbine energy for a day. Power rises with the cube of speed, so computing from the daily mean alone underestimates energy. The function assumes speeds within the day follow a Rayleigh distribution around the mean, and averages the power curve over it. This is an assumption: NASA daily data has no hourly speeds.

Tests that run

- Zero sunlight gives zero panel energy.
- The lamp battery stays between its floor and its capacity.
- More sunlight never gives fewer lit hours.

- Turbine power is 0 below cut-in, equals rated power at rated speed, and is 0 above cut-out.
- For a pure cubic curve, the Rayleigh average is about 1.91 times the cube of the mean speed (a known property; the test catches a wrong formula).
- Hot-water temperature stays between the air temperature and the safe maximum.

Look at one site. Save the snippet in the Code section as `one_site.py`, run `python one_site.py`, and you get two figures in `results/`: the lamp battery over a year with hot-water temperature by month, and the turbine kWh by month. Use them on the device slides, labelled "simulated".

What to write for the slides

- Number of nights the lamp fails and in which months (read it from the plot).
- Which months need heater backup.
- Which month gives the most turbine kWh.
- One line each on what the formula assumes. Don't say "measured".

Step 4: four EDA plots and the inference under each (15 min)

`run_all.py` writes four plots to `results/` and prints the numbers you need. For each one, write two lines for the slide: **What the data shows** (a number you read off the plot) and **Inference** (the decision it leads to for a device or a feature). The blanks below are for you to fill from the real plots; I have not seen your real data, so I have not guessed the answers.

File	What it shows	What to read off it	Inference to write (fill the blanks)
`eda1_sunlight_by_month.png`	monthly sunlight at the 4 solar sites	lowest and highest month and their values (printed by the script)	"Sunlight is lowest in month __, so season sin and cos go into the sunlight model, and the lamp and heater need a plan for those months."
`eda2_wind_by_month.png`	wind at 10 m and 50 m at the 4 wind sites	windiest and calmest month; how much 50 m exceeds 10 m	"Wind peaks in month __ and speed rises with height by about __ times, so we use the 50 m reading and the shear to get hub-height speed."

File	What it shows	What to read off it	Inference to write (fill the blanks)
`eda3_cloudy_streaks.png`	number of runs of 3 or more cloudy days, and the longest run, per solar site	the two numbers per site (also in `eda3_cloudy_streaks.csv`)	"Cloudy runs of 3 or more days happen __ times per site and the longest is __ days, so the lamp needs 3-day sunlight history and a streak-length feature."
`eda4_solar_vs_wind.png`	monthly solar and wind relative to their own average	whether the two curves move in opposite directions	"Solar is lowest when wind is highest (or not), so a wind-solar hybrid suggestion makes sense (or does not)."

Extra numbers to record on the slides

- The cloudy-day cut and the low-wind cut are each site's own training-period percentiles (25th for sunlight, 20th for hub-height wind). State this on the slide.
- Missing values: how many NaN per column, and that rows next to a gap are dropped from the models, never filled.

Checks before you trust a plot

- Every curve has 12 months and starts and ends where you expect.

- Units are on the axis (kWh per m2 per day, m/s).
- If a site looks very different from the others, look at its raw rows before explaining it.
- If the real shape contradicts the "expected" inference above, write what the data shows and change the inference; don't force it.

Step 5: device-specific features and two quick models (30 min)

This is the part that answers the faculty's "features are too generic" point. Each device has its own inputs, chosen for the physics of that device, and its own target. `run_all.py` builds both feature sets and trains the models.

Device	Model target (tomorrow)	Features	Why these
Solar lamp and heater	sunlight at a solar site (kWh per m2 per day)	sunlight today, 3-day mean, 3-day minimum, cloudy-streak length, season sin and cos, air temperature, wind at 10 m	the battery and tank carry the memory of recent days; a streak drains the battery; air temperature and wind set panel and tank losses
Wind turbine	wind speed at hub height at a wind site (m/s)	wind at 10 m and 50 m, shear, hub-height speed, its cube, yesterday's speed, 3-day mean speed, air temperature, season sin and cos	power follows the cube of speed; shear gets speed to hub height; the cube lets a simple model see the power effect

How the models are set up

- One pooled model per target across its 4 sites, with a site indicator.
- "Tomorrow" means features of day t and the label of day t+1. Features use only days up to t.
- Planners compared: **naive** (the site's training average every day), **seasonal** (training average for that site and month), **persistence** (today's value), **Ridge**, and **Histogram Gradient Boosting** (seed 42).
- Fit on train (to 2023). Look at validation (2024) to choose; report test (2025 onward) once. The script prints both.

- Output: `results/model_results.csv` with MAE and the ratio to naive. Below 1.00 beats naive.

How to read the result

- If Ridge or Gradient Boosting beats persistence and seasonal on validation and test, say by how much (ratio and MAE in kWh per m2 per day or m/s).
- If persistence or the seasonal average wins, say that. It is a valid result; without cloud forecasts, tomorrow's sunlight is hard to beat with a seasonal average.
- If the validation and test rankings disagree, say so; don't pick the test winner.
- Do not tune the models today. Two fixed models are enough for Phase 1.

Fixes if it fails. A `KeyError` on a column means a name mismatch in the loader. A shape error in the models means too many rows were dropped for NaN, so print `len(sun_df)` and `len(wind_df)`. An `inf` error means a zero wind reading; the script already turns it into NaN, so check which site has a zero in `WS10M`.

Optional (10 min, only if time): log each row of `model_results.csv` to MLflow as a run, with tags for the git commit and the data file names, so the MLOps slide can show real device-model runs. Don't skip the results for this.

Step 6: device error, the site scorecard and the awareness messages (20 min)

Device error (`results/device_error.csv`). For each planner, the script feeds its tomorrow's sunlight or wind estimate into the device function and compares the output with the one computed from the actual weather, on the test period.

- Lamp: MAE of lit hours, and precision and recall of the "lamp fails tonight" flag.
- Heater: MAE of hot-water temperature in degrees C, and precision and recall of "backup needed".
- Turbine: MAE of kWh per day.
- Columns ending `_ratio_to_naive` divide each planner's error by the naive planner's. Below 1.00 is better.

What the script holds fixed. Tomorrow's air temperature and wind at 10 m are taken from the actual weather; only the sunlight (lamp and heater) or hub-height wind (turbine) is estimated. Say this on the slide. The lamp battery carries over from day to day, using the

days that exist in the test period; rows dropped for missing values slightly break the chain. Also say that precision and recall are shown because failures are rare; accuracy would look good for a planner that never predicts failure.

Site scorecard (`results/site_scorecard.csv`). For each of the 8 sites, using all days of actual weather, the script gives:

- the share of nights the lamp fails,
- the share of days the heater needs backup,
- turbine kWh per month and turbine capacity factor (daily kWh divided by 24 x rated kW),
- the share of low-wind days (hub-height speed below the site's training 20th percentile).

Recommendations. `REC` at the top of `run_all.py` holds the thresholds: lamp fails on fewer than 5 percent of nights, heater backup on fewer than 30 percent of days, turbine capacity factor at least 15 percent and low-wind days at most 35 percent. These are design choices. **Fix them before you look at the scorecard** and tell Minnie they are choices, not standards. Do not change them to get a nicer-looking answer. The script prints one line per site, for example "At <site>: solar lamp recommended (fails __% of nights); small turbine not recommended (CF __%)." Use these as the sample awareness messages on a slide.

Awareness message template (tomorrow) for one slide mock-up, filled from your output:

- Lamp: "Tomorrow's sunlight is low and the last __ days were cloudy. Your lamp may light about __ hours tonight."
- Heater: "Expect about __ degrees C hot water tomorrow. Plan the electric backup."
- Turbine: "Low-wind day expected. Expect about __ kWh instead of your usual __."

Take the blanks from one real test day: print one row of `sun_df` or `wind_df`, run its estimate through the device function, and use those numbers. Don't invent them.

Limits to write on the slides

- Weather is satellite and reanalysis data, not a ground station next to the device.
- Sunlight is for a flat surface; tilt, shade and dust are not modelled.
- The recommendation is resource suitability only; it ignores cost, space and installation.
- Eight sites only; other districts need more NASA pulls.

Hand-back: what to send and the honesty rules

Send all of this to Minnie (and the results files to Claude if she asks for help with slide text):

- [ ] `results/` folder: the four `eda*.png`, `dev1_lamp_heater.png`, `dev2_turbine_monthly.png`, `model_results.csv`, `device_error.csv`, `site_scorecard.csv`, `eda3_cloudy_streaks.csv`
- [ ] `device_specs.csv` with sources
- [ ] the console output of `run_all.py` (copy it into a text file)
- [ ] `RESULTS.md` (template below)
- [ ] the branch name with the code, and whether `python devices.py` passes

RESULTS.md template

```
# Results (simulated devices, real NASA weather)
Data: __ sites, __ rows, __ missing values; train to 2023, val 2024, test
2025+.
Specs: verified = yes/no; assumptions listed in device_specs.csv.

## Models (tomorrow, test period)
Sunlight: best planner = __, MAE __ kWh/m2/day, ratio to naive __, ratio to
persistence __.
Hub-height wind: best planner = __, MAE __ m/s, ratio to naive __, ratio to
persistence __.
Validation and test rankings agree: yes/no.

## Devices (test period)
Lamp: MAE __ h; fail-flag precision __ recall __ (best planner).
Heater: MAE __ C; backup-flag precision __ recall __.
Turbine: MAE __ kWh/day.

## Scorecard
(paste the per-site lines printed by run_all.py)

## What went wrong or looks odd
- __

## Not done
- __
```

Honesty rules

- Say "simulated" for every device number. Say "estimated" for model output.
- Report results even when the simple planners win.
- Never report test numbers from a model you chose by looking at test.
- If a number looks too good, find the cause before sending it. A common cause is a feature that uses tomorrow's data; the features here use up to today only.
- Anything you could not do goes under "Not done". Minnie will put it on the next-steps slide, so it is better to say it than to hide it.

Code

Three files, all in one folder with the data folder next to them. Python 3 with numpy, pandas, scikit-learn and matplotlib. I ran these on made-up NASA-format data (8 sites, 2,528 days each, a few -999 values): they ran start to finish and the physics tests pass. They have not run on your real files, and the placeholder specs are not real products.

1. **`devices.py`** (device formulas and physics tests; run `python devices.py`)

```
"""Device formulas. Output is SIMULATED from formulas, not measured."""
import numpy as np
import pandas as pd

SPECS_VERIFIED = False  # set True only after every value below has a source
in device_specs.csv

SPECS = {
    "lamp": dict(panel_W=30.0, battery_Wh=100.0, bulb_W=8.0,
hours_wanted=10.0,
                 system_eff=0.75, charge_eff=0.85, depth_of_discharge=0.8,
                 temp_loss=0.004, temp_rise=20.0),
    "heater": dict(area_m2=2.0, tank_litres=100.0, collector_eff=0.5,
                   UL0=1.5, UL1=0.1, setpoint_C=45.0, T_max_C=90.0),
    "turbine": dict(rated_kW=3.0, cut_in=3.0, rated_speed=11.0, cut_out=25.0,
                    hub_height_m=20.0),
}


def warn_if_placeholder():
    if not SPECS_VERIFIED:
        print("WARNING: PLACEHOLDER SPECS. Replace SPECS with sourced values
before using any number.")
```

```
# ---------- solar lamp ----------
def panel_energy_wh(sun_hours, temp_c, s):
    t_cell = temp_c + s["temp_rise"]
    derate = 1.0 - s["temp_loss"] * (t_cell - 25.0)
    return s["panel_W"] * sun_hours * s["system_eff"] * derate


def lamp_simulate(sun, temp, s=None):
    """sun, temp: daily arrays in date order. Returns DataFrame with
hours_lit, soc_end_wh, fails."""
    s = s or SPECS["lamp"]
    sun = np.asarray(sun, float)
    temp = np.asarray(temp, float)
    floor = s["battery_Wh"] * (1 - s["depth_of_discharge"])
    soc_prev = s["battery_Wh"]
    hours, soc_end = [], []
    for k in range(len(sun)):
        e = panel_energy_wh(max(sun[k], 0.0), temp[k], s) * s["charge_eff"]
        charged = min(s["battery_Wh"], soc_prev + max(e, 0.0))
        h = min(s["hours_wanted"], max(0.0, (charged - floor) / s["bulb_W"]))
        soc_prev = charged - s["bulb_W"] * h
        hours.append(h)
        soc_end.append(soc_prev)
    out = pd.DataFrame({"hours_lit": hours, "soc_end_wh": soc_end})
    out["fails"] = out["hours_lit"] < s["hours_wanted"] - 1e-9
    return out


# ---------- solar water heater ----------
def heater_simulate(sun, temp, wind10, s=None):
    s = s or SPECS["heater"]
    sun = np.asarray(sun, float)
    temp = np.asarray(temp, float)
    wind10 = np.asarray(wind10, float)
    q_gain = s["area_m2"] * s["collector_eff"] * np.maximum(sun, 0.0)
# kWh
    q_loss = (s["UL0"] + s["UL1"] * wind10) * (s["setpoint_C"] - temp) * 24 /
1000  # kWh, tank at setpoint
    q_loss = np.maximum(q_loss, 0.0)
    dT = (q_gain - q_loss) * 3600 / (s["tank_litres"] * 4.186)
    t_hot = np.clip(temp + dT, temp, s["T_max_C"])
    return pd.DataFrame({"t_hot_C": t_hot, "backup_needed": t_hot <
```

```
s["setpoint_C"]})


# ---------- wind turbine ----------
def hub_speed(ws10, ws50, hub_height_m):
    ws10 = np.asarray(ws10, float)
    ws50 = np.asarray(ws50, float)
    alpha = np.log(ws50 / ws10) / np.log(5.0)
    return ws50 * (hub_height_m / 50.0) ** alpha


def power_kw(v, s=None):
    s = s or SPECS["turbine"]
    v = np.asarray(v, float)
    ramp = s["rated_kW"] * (v**3 - s["cut_in"]**3) / (s["rated_speed"]**3 -
s["cut_in"]**3)
    p = np.where(v < s["cut_in"], 0.0, np.where(v < s["rated_speed"], ramp,
s["rated_kW"]))
    return np.where(v > s["cut_out"], 0.0, p)


def daily_energy_kwh(v_mean, s=None, hours=24):
    """Average the power curve over a Rayleigh distribution with the given
daily mean speed."""
    s = s or SPECS["turbine"]
    v_mean = np.atleast_1d(np.asarray(v_mean, float))
    v = np.linspace(0.05, 40, 800)
    dv = v[1] - v[0]
    p = power_kw(v, s)
    out = np.zeros(len(v_mean))
    for i, m in enumerate(v_mean):
        if not np.isfinite(m) or m <= 0:
            out[i] = np.nan if not np.isfinite(m) else 0.0
            continue
        scale = m / np.sqrt(np.pi / 2)
        pdf = (v / scale**2) * np.exp(-v**2 / (2 * scale**2))
        out[i] = hours * float(np.sum(p * pdf * dv))
    return out


# ---------- physics tests (run: python devices.py) ----------
def run_tests():
    s = SPECS["lamp"]
```

```
    assert panel_energy_wh(0.0, 30.0, s) == 0
    rng = np.random.default_rng(0)
    out = lamp_simulate(rng.uniform(0, 7, 400), np.full(400, 28.0))
    floor = s["battery_Wh"] * (1 - s["depth_of_discharge"])
    assert (out.soc_end_wh >= floor - 1e-9).all() and (out.soc_end_wh <=
s["battery_Wh"] + 1e-9).all()
    lo = lamp_simulate(np.full(30, 2.0), np.full(30, 28.0)).hours_lit
    hi = lamp_simulate(np.full(30, 5.0), np.full(30, 28.0)).hours_lit
    assert (hi >= lo - 1e-9).all()
    t = SPECS["turbine"]
    assert power_kw(t["cut_in"] - 0.1) == 0
    assert abs(float(power_kw(t["rated_speed"])) - t["rated_kW"]) < 1e-9
    assert power_kw(t["cut_out"] + 1) == 0
    # Rayleigh check: for a cubic power curve energy ~ 1.91 x cube of the
mean
    cubic = lambda v: v**3
    m = 4.0
    scale = m / np.sqrt(np.pi / 2)
    v = np.linspace(0.01, 60, 6000)
    pdf = (v / scale**2) * np.exp(-v**2 / (2 * scale**2))
    ratio = np.sum(cubic(v) * pdf * (v[1] - v[0])) / m**3
    assert abs(ratio - 1.91) < 0.02, ratio
    h = heater_simulate([0.0, 6.0], [25.0, 25.0], [2.0, 2.0])
    assert (h.t_hot_C >= 25.0 - 1e-9).all() and (h.t_hot_C <= SPECS["heater"]
["T_max_C"]).all()
    assert h.t_hot_C.iloc[1] > h.t_hot_C.iloc[0]
    print("all device tests passed")


if __name__ == "__main__":
    warn_if_placeholder()
    run_tests()
```

2. **`one_site.py`** (quick figures for the device slides; run after `run_all.py` works; it re-runs `run_all.py` once on import)

```
import matplotlib; matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import devices as D
import run_all as R

SOLAR, WIND, YEAR = "Pavagada", "Chitradurga", 2025
```

```
g = R.d[(R.d.site == SOLAR) & (R.d.date.dt.year == YEAR)].dropna(subset=
["sun", "T2M", "WS10M"]).reset_index(drop=True)
lamp = D.lamp_simulate(g.sun, g.T2M)
heat = D.heater_simulate(g.sun, g.T2M, g.WS10M)
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].plot(g.date, lamp.soc_end_wh); ax[0].set_title(f"Lamp battery at
{SOLAR}, {YEAR} (simulated)"); ax[0].set_ylabel("Wh")
heat.assign(month=g.date.dt.month).groupby("month").t_hot_C.mean().plot(ax=ax
[1], marker="o")
ax[1].axhline(D.SPECS["heater"]["setpoint_C"], ls="--", c="grey");
ax[1].set_title(f"Hot-water temperature by month at {SOLAR} (simulated)")
plt.tight_layout(); plt.savefig(R.OUT / "dev1_lamp_heater.png", dpi=150);
plt.close()
print(f"lamp fails on {lamp.fails.sum()} of {len(lamp)} nights; heater backup
on {heat.backup_needed.sum()} days")

w = R.d[(R.d.site == WIND) & (R.d.date.dt.year == YEAR)].dropna(subset=
["v_hub"]).reset_index(drop=True)
kwh = D.daily_energy_kwh(w.v_hub.to_numpy())
w.assign(kwh=kwh,
month=w.date.dt.month).groupby("month").kwh.sum().plot(kind="bar")
plt.title(f"Turbine kWh by month at {WIND}, {YEAR} (simulated)");
plt.ylabel("kWh")
plt.tight_layout(); plt.savefig(R.OUT / "dev2_turbine_monthly.png", dpi=150);
plt.close()
```

3. **`run_all.py`** (loads data, EDA plots, two models, device error, scorecard; edit the config block at the top, then run `python run_all.py`)

```
"""Load NASA data, make EDA plots, train two quick models, score devices,
write a site scorecard.
Run: python run_all.py     (needs devices.py in the same folder)"""
import glob
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import Ridge
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
```

```
import devices as D

# ---------------- config: edit these ----------------
NASA_GLOB = "data/raw/nasa/*.csv"          # one file per site; file name
(without .csv) = site name
SOLAR_SITES = ["Pavagada", "Raichur", "Kalaburagi", "Mysuru"]
WIND_SITES = ["Chitradurga", "Gadag", "Koppal", "Hassan"]
OUT = Path("results")
TRAIN_END, VAL_END = "2023-12-31", "2024-12-31"   # train <= 2023, val =
2024, test >= 2025
REC = dict(lamp_fail_max=0.05, heater_backup_max=0.30, turbine_cf_min=0.15,
low_wind_max=0.35)  # design choices: fix BEFORE looking
# ----------------------------------------------------
OUT.mkdir(exist_ok=True)
D.warn_if_placeholder()


def load_nasa(pattern):
    frames = []
    for p in sorted(glob.glob(pattern)):
        lines = Path(p).read_text().splitlines()
        i = next(k for k, l in enumerate(lines) if l.startswith("YEAR"))
        df = pd.read_csv(p, skiprows=i)
        df["date"] = pd.to_datetime(dict(year=df.YEAR, month=df.MO,
day=df.DY))
        df["site"] = Path(p).stem
        frames.append(df[["site", "date", "ALLSKY_SFC_SW_DWN", "T2M",
"WS10M", "WS50M"]])
    d = pd.concat(frames).replace(-999, np.nan).replace(-999.0, np.nan)
    d = d.rename(columns={"ALLSKY_SFC_SW_DWN": "sun"}).sort_values(["site",
"date"]).reset_index(drop=True)
    return d


def split_of(dates):
    return np.where(dates <= pd.Timestamp(TRAIN_END), "train", np.where(dates
<= pd.Timestamp(VAL_END), "val", "test"))


# ---------------- step 1: load and check ----------------
d = load_nasa(NASA_GLOB)
d["split"] = split_of(d["date"])
print("rows per site:\n", d.groupby("site").size())
```

```
print("NaN per column:\n", d[["sun", "T2M", "WS10M", "WS50M"]].isna().sum())
for site, g in d.groupby("site"):
    assert (g.date.diff().dropna() == pd.Timedelta(days=1)).all(), f"date gap
or duplicate at {site}"
assert (d.sun.dropna() >= 0).all(), "negative sunlight"
hub_h = D.SPECS["turbine"]["hub_height_m"]
d["v_hub"] = D.hub_speed(d.WS10M, d.WS50M, hub_h)
d["v_hub"] = d["v_hub"].replace([np.inf, -np.inf], np.nan)   # a zero wind
reading would give inf; treat as missing
d["month"] = d.date.dt.month

# ---------------- step 2: EDA plots ----------------
thr_sun = d[d.split == "train"].groupby("site").sun.quantile(0.25)   # cloudy
day = below the site's train 25th percentile
thr_low = d[d.split == "train"].groupby("site").v_hub.quantile(0.20)  # low-
wind day = below the site's train 20th percentile

m_sun = d[d.site.isin(SOLAR_SITES)].groupby(["month",
"site"]).sun.mean().unstack()
m_sun.plot(marker="o"); plt.ylabel("sunlight (kWh/m2/day)");
plt.xlabel("month"); plt.title("Sunlight by month, solar sites")
plt.tight_layout(); plt.savefig(OUT / "eda1_sunlight_by_month.png", dpi=150);
plt.close()

w = d[d.site.isin(WIND_SITES)]
fig, ax = plt.subplots(1, 2, figsize=(10, 4), sharey=True)
w.groupby(["month", "site"]).WS10M.mean().unstack().plot(ax=ax[0],
marker="o", title="Wind at 10 m")
w.groupby(["month", "site"]).WS50M.mean().unstack().plot(ax=ax[1],
marker="o", title="Wind at 50 m")
ax[0].set_ylabel("m/s"); plt.tight_layout(); plt.savefig(OUT /
"eda2_wind_by_month.png", dpi=150); plt.close()


def streaks(flag):
    runs, c = [], 0
    for f in flag:
        if f:
            c += 1
        elif c:
            runs.append(c); c = 0
    if c:
        runs.append(c)
```

```
    return runs


rows = []
for s in SOLAR_SITES:
    g = d[d.site == s]
    r = streaks((g.sun < thr_sun[s]).tolist())
    rows.append(dict(site=s, runs_ge3=sum(x >= 3 for x in r), longest=max(r)
if r else 0))
streak_tbl = pd.DataFrame(rows).set_index("site")
streak_tbl.plot(kind="bar"); plt.title("Cloudy streaks (sun below site 25th
percentile)")
plt.tight_layout(); plt.savefig(OUT / "eda3_cloudy_streaks.png", dpi=150);
plt.close()
streak_tbl.to_csv(OUT / "eda3_cloudy_streaks.csv")

sun_m = d[d.site.isin(SOLAR_SITES)].groupby("month").sun.mean()
wind_m = d[d.site.isin(WIND_SITES)].groupby("month").v_hub.mean()
pd.DataFrame({"solar (relative)": sun_m / sun_m.mean(), "wind hub speed
(relative)": wind_m / wind_m.mean()}).plot(marker="o")
plt.axhline(1, ls="--", c="grey"); plt.title("Solar and wind by month,
relative to own mean")
plt.tight_layout(); plt.savefig(OUT / "eda4_solar_vs_wind.png", dpi=150);
plt.close()
print("\nEDA numbers for the slide text:")
print("lowest sunlight month:", int(sun_m.idxmin()), round(sun_m.min(), 2),
"highest:", int(sun_m.idxmax()), round(sun_m.max(), 2))
print("windiest month:", int(wind_m.idxmax()), round(wind_m.max(), 2),
"calmest:", int(wind_m.idxmin()), round(wind_m.min(), 2))
print(streak_tbl)

# ---------------- step 3: features (device specific) and two quick models --
--------------
def sun_frame(g):
    g = g.copy()
    g["sun_mean3"] = g.sun.rolling(3).mean()
    g["sun_min3"] = g.sun.rolling(3).min()
    cloudy = (g.sun < thr_sun[g.site.iloc[0]]).astype(int)
    g["cloud_streak"] = cloudy.groupby((cloudy == 0).cumsum()).cumsum()
    ang = 2 * np.pi * g.date.dt.dayofyear / 365.25
    g["season_sin"], g["season_cos"] = np.sin(ang), np.cos(ang)
    g["today"] = g.sun
    g["target"] = g.sun.shift(-1)
```

```
    g["target_date"] = g.date.shift(-1)
    return g


def wind_frame(g):
    g = g.copy()
    g["shear"] = (np.log(g.WS50M / g.WS10M) / np.log(5.0)).replace([np.inf, -
np.inf], np.nan)
    g["v_cube"] = g.v_hub ** 3
    g["v_lag1"] = g.v_hub.shift(1)
    g["v_mean3"] = g.v_hub.rolling(3).mean()
    ang = 2 * np.pi * g.date.dt.dayofyear / 365.25
    g["season_sin"], g["season_cos"] = np.sin(ang), np.cos(ang)
    g["today"] = g.v_hub
    g["target"] = g.v_hub.shift(-1)
    g["target_date"] = g.date.shift(-1)
    return g


SUN_FEATS = ["sun", "sun_mean3", "sun_min3", "cloud_streak", "season_sin",
"season_cos", "T2M", "WS10M"]
WIND_FEATS = ["WS10M", "WS50M", "shear", "v_hub", "v_cube", "v_lag1",
"v_mean3", "T2M", "season_sin", "season_cos"]


def planners(df, feats):
    """Return df (rows with all features and target) plus one prediction
column per planner."""
    df = df.dropna(subset=feats + ["target", "target_date"]).copy()
    df["tmonth"] = df.target_date.dt.month
    tr = df[df.split == "train"]
    df["naive"] = df.site.map(tr.groupby("site").target.mean())
    seas = tr.groupby(["site",
"tmonth"]).target.mean().rename("seasonal").reset_index()
    df = df.merge(seas, on=["site", "tmonth"], how="left")
    df["persistence"] = df["today"]
    X = pd.concat([df[feats], pd.get_dummies(df.site, dtype=float)], axis=1)
    tr_idx = df.split == "train"
    models = {"ridge": make_pipeline(StandardScaler(), Ridge(alpha=1.0)),
              "hgb": HistGradientBoostingRegressor(random_state=42,
max_iter=300, learning_rate=0.05)}
    for name, m in models.items():
        m.fit(X[tr_idx], df.loc[tr_idx, "target"])
```

```
        df[name] = m.predict(X)
    return df


PL = ["naive", "seasonal", "persistence", "ridge", "hgb"]


def mae_table(df, label):
    rows = []
    for split in ["val", "test"]:
        part = df[df.split == split]
        base = (part.naive - part.target).abs().mean()
        for p in PL:
            mae = (part[p] - part.target).abs().mean()
            rows.append(dict(task=label, split=split, planner=p, mae=mae,
ratio_to_naive=mae / base, n=len(part)))
    return pd.DataFrame(rows)


sun_df = planners(pd.concat([sun_frame(g) for _, g in
d[d.site.isin(SOLAR_SITES)].groupby("site")]), SUN_FEATS)
wind_df = planners(pd.concat([wind_frame(g) for _, g in
d[d.site.isin(WIND_SITES)].groupby("site")]), WIND_FEATS)
res = pd.concat([mae_table(sun_df, "sunlight_tomorrow (kWh/m2/day)"),
mae_table(wind_df, "hub_wind_tomorrow (m/s)")])
res.to_csv(OUT / "model_results.csv", index=False)
print("\nModel results (ratio < 1 beats naive):\n",
res.round(3).to_string(index=False))

# ---------------- step 4: device error: estimate vs actual weather ---------
-------
def pr(actual, pred):
    tp = int((actual & pred).sum())
    return tp / max(int(pred.sum()), 1), tp / max(int(actual.sum()), 1)


dev_rows = []
actual_by_key = d.set_index(["site", "date"])
test = sun_df[sun_df.split == "test"].sort_values(["site", "target_date"])
for p in PL:
    err_l, err_h, fl_a, fl_p, bk_a, bk_p = [], [], [], [], [], []
    for site, g in test.groupby("site"):
        w_ = actual_by_key.loc[[(site, t) for t in g.target_date]]   #
```

```
tomorrow's actual T2M and WS10M
        t2, ws = w_["T2M"].to_numpy(), w_["WS10M"].to_numpy()
        ok = ~(np.isnan(t2) | np.isnan(ws))
        a_l = D.lamp_simulate(g.target.to_numpy()[ok], t2[ok]); e_l =
D.lamp_simulate(g[p].to_numpy()[ok], t2[ok])
        a_h = D.heater_simulate(g.target.to_numpy()[ok], t2[ok], ws[ok]); e_h
= D.heater_simulate(g[p].to_numpy()[ok], t2[ok], ws[ok])
        err_l += list((a_l.hours_lit - e_l.hours_lit).abs()); err_h +=
list((a_h.t_hot_C - e_h.t_hot_C).abs())
        fl_a += list(a_l.fails); fl_p += list(e_l.fails); bk_a +=
list(a_h.backup_needed); bk_p += list(e_h.backup_needed)
    pl, rl = pr(np.array(fl_a), np.array(fl_p)); ph, rh = pr(np.array(bk_a),
np.array(bk_p))
    dev_rows.append(dict(planner=p, lamp_mae_hours=np.mean(err_l),
lamp_fail_precision=pl, lamp_fail_recall=rl,
                         heater_mae_C=np.mean(err_h), backup_precision=ph,
backup_recall=rh))
wt = wind_df[wind_df.split == "test"]
for row in dev_rows:
    p = row["planner"]
    row["turbine_mae_kwh_day"] =
np.nanmean(np.abs(D.daily_energy_kwh(wt.target.to_numpy()) -
D.daily_energy_kwh(wt[p].to_numpy())))
dev = pd.DataFrame(dev_rows)
for c in ["lamp_mae_hours", "heater_mae_C", "turbine_mae_kwh_day"]:
    dev[c + "_ratio_to_naive"] = dev[c] / dev.loc[dev.planner == "naive",
c].iloc[0]
dev.to_csv(OUT / "device_error.csv", index=False)
print("\nDevice error on the test period (simulated):\n",
dev.round(3).to_string(index=False))

# ---------------- step 5: site scorecard and recommendations ---------------
-
cards = []
for site, g in d.groupby("site"):
    g = g.dropna(subset=["sun", "T2M", "WS10M", "WS50M"])
    lamp = D.lamp_simulate(g.sun, g.T2M)
    heat = D.heater_simulate(g.sun, g.T2M, g.WS10M)
    kwh = D.daily_energy_kwh(g.v_hub.to_numpy())
    cards.append(dict(site=site, lamp_fail_share=lamp.fails.mean(),
heater_backup_share=heat.backup_needed.mean(),
                      turbine_kwh_per_month=np.nanmean(kwh) * 30.4,
                      turbine_cf=np.nanmean(kwh) / (24 * D.SPECS["turbine"]
```

```
["rated_kW"]),
                       low_wind_share=(g.v_hub < thr_low[site]).mean()))
card = pd.DataFrame(cards).set_index("site")


def advise(r):
    out = []
    out.append(f"solar lamp {'recommended' if r.lamp_fail_share <
REC['lamp_fail_max'] else 'not recommended'} (fails {r.lamp_fail_share:.0%}
of nights)")
    out.append(f"solar heater {'recommended' if r.heater_backup_share <
REC['heater_backup_max'] else 'needs backup'} (backup on
{r.heater_backup_share:.0%} of days)")
    ok = r.turbine_cf >= REC["turbine_cf_min"] and r.low_wind_share <=
REC["low_wind_max"]
    out.append(f"small turbine {'recommended' if ok else 'not recommended'}
(CF {r.turbine_cf:.0%}, low-wind {r.low_wind_share:.0%} of days)")
    return "; ".join(out)


card["advice"] = card.apply(advise, axis=1)
card.to_csv(OUT / "site_scorecard.csv")
print("\nSite scorecard (simulated):\n",
card.drop(columns="advice").round(3))
for s, r in card.iterrows():
    print(f"At {s}: {r.advice}.")
D.warn_if_placeholder()
print("\nDone. Files are in", OUT.resolve())
```